

Let's Talk Tempo: Variable Speech Rate Conversational Agents

Amritha Iyer
amrithaiyer02@utexas.edu
University of Texas at Austin
United States

Abhinav Jayaraman
aj29927@utexas.edu
University of Texas at Austin
United States

Abstract

Despite strides in making Conversational Agents (CAs) more human-like through pitch variation, many users still find such agents difficult to communicate with due to assumptions made by developers on what "normal" speech patterns are. Rate of speech is one such variable that remains inflexible in most CAs despite its role in governing the comprehension and efficiency of interpersonal communication. We aim to implement and test a dynamic conversational agent that adapts to the user's rate of speech. The findings will reveal whether changing the speech rate of a CAs to match that of the speaker impacts the user experience, evaluated for both comprehension and efficacy.

ACM Reference Format:

Amritha Iyer and Abhinav Jayaraman. 2025. Let's Talk Tempo: Variable Speech Rate Conversational Agents. In . ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Goal

Currently, Commercial Conversational Agents are often calibrated singularly to an "average" user in terms of their rate of speech. We seek to test our hypothesis that the use of an adaptive Speech Rate (SR) CA that adjusts based on the user's SR will improve comprehension and efficiency.

2 Related Work

For English, four main paralinguistic aspects or adjustable elements of spoken speech are identified: speech rate, volume, pitch, and timbre[3]. In the context of interacting with CAs, these factors can be modulated from interaction to interaction as best suited to the needs of the speaker without inherently modifying the content. Variations in speaking rate (SR) preferences can be seen across different speaker populations. A Native speaker (NS) and non-native speaker (NNS) of a language, for example, are impacted differently by the rate of speech when it comes to speech and listening comprehension. Preliminary experiments by Inoue and Liao to assess the impact of speech rate (SR) on comprehension revealed that of the 115 incomprehensible speech segments recorded in 150 minutes of conversation, 24 cases were due to the SR of the NS being too fast, comprising 21% of all cases [4]. When examining how such factors impact interaction with speech synthesizing systems, researchers

evaluating the use of CAs by brands found that consumers who used digital assistance with a fast or slow SR were less likely to want to use voice-assistive technology than consumers who interacted with a voice agent who used a "moderate" SR [3].

Another study by Choi et al. showed that manipulating the speech rate could impact the user's impressions of the agent. Users felt that the agent with base SR was more "human-like" and intimate, especially evident in their engagement in small talk with the agent. In contrast, users described the faster-speaking agent as "mechanical" and found it frustrating when the more rapid SR led to missed information. Accordingly, users tended to interact with the faster-speaking agent using almost exclusively functional commands. [2].

Another factor to consider when increasing the accessibility and efficacy of dialogue is that people are often able to comprehend synthetic speech at a faster rate than they can speak it themselves. Bragg et al. found that this is especially true for users who have been exposed to synthetic speech from an earlier age. These users may find it more efficient to communicate with a CA that speaks faster than themselves [1].

3 Contribution

The existing studies on voice agents described above have been limited to manipulating the agent's rate of speech at predetermined scales. Choi et al. exclusively tested an increased SR against a baseline, with findings that were primarily applied to compare the preferences of blind/low-vision users of the voice agent and fully sighted users.

Rather than providing unidirectional or static adjustments to speaking rate (i.e. 1.75x speed) in the way prior research has, our aim is to implement a proof-of-concept for an adaptive CA, in a small user study for feedback and impressions. By comparing the feedback and data to the default SR of the system, this paper will help determine if the adaptive SR of the agent is positively or negatively impacting the user experience and listening comprehension, or if there is no notable difference.

4 Implementation

As outlined in Figure 1, we first identified a well-established means of speech-to-text transcription as well as speech rate detection. This manifested in using industry standard speech-to-text systems and PRAAT scripts from the Myprosody package and repository. Our Python notebook acts as a wrapper for our chosen LLM. Throughout this section, we will also discuss various challenges that were encountered in this section for the two approaches taken, followed by details of shared components between them.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

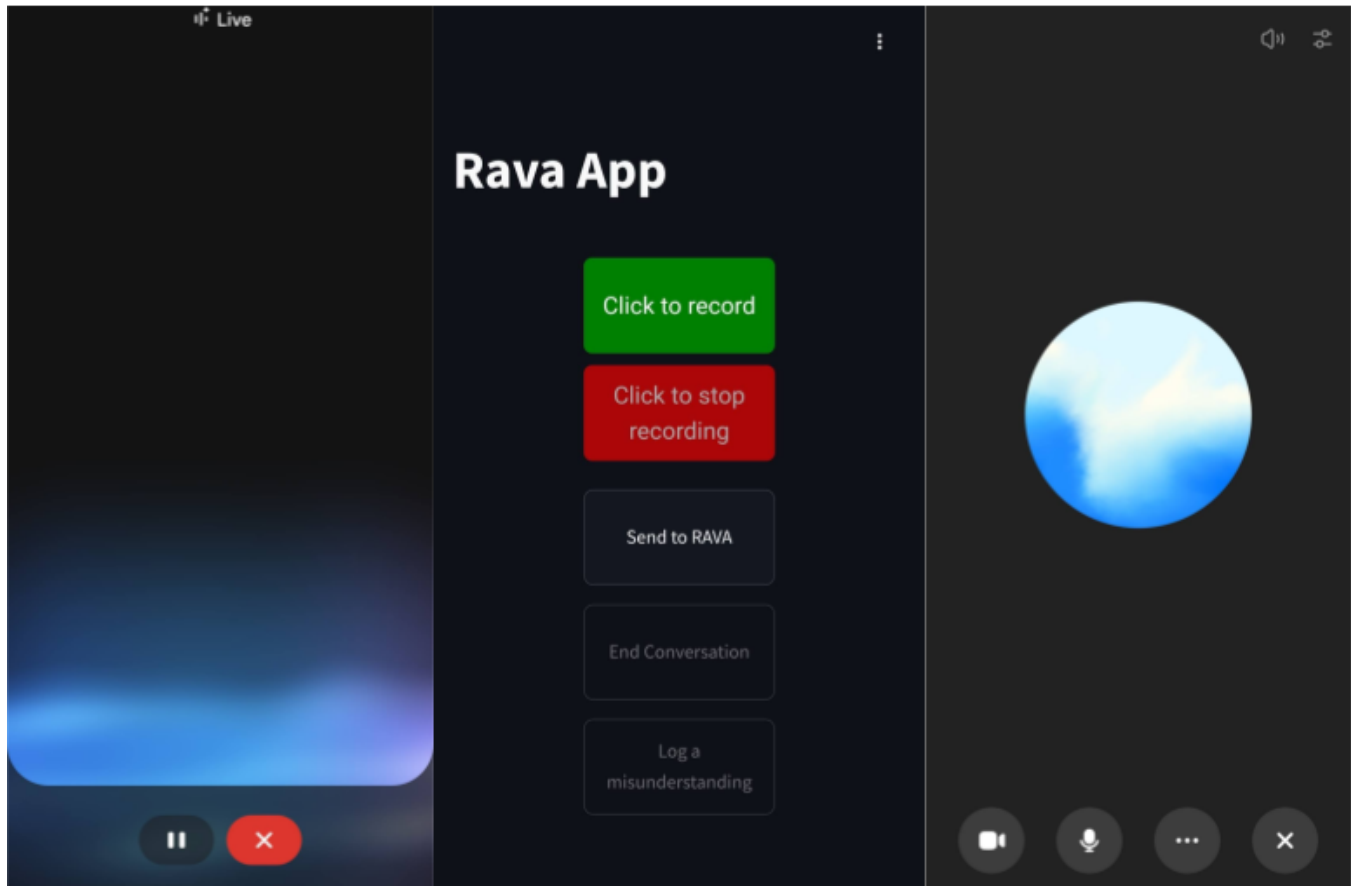


Figure 1: From left to right: Gemini Live, Our web-app approach, RAVA <https://github.com/i-am-ami/hci-rava-project/tree/deployable>, and ChatGPT Advanced Voice

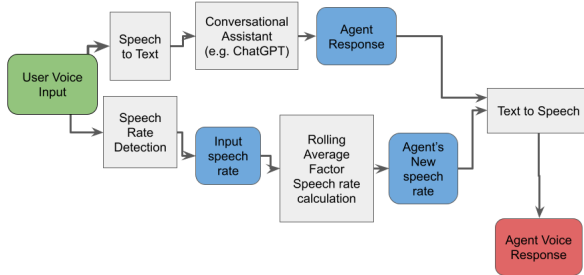


Figure 2: Sketch of our adaptive CA's data pipeline. Green is for User Input, Blue for Intermediate Data, Red for Output, Square Boxes denote Systems/Programs, and the Rounded Boxes denote Data.

4.1 Local Approach

While a local approach ended up as our proof-of concept, the task of setting up our environment and figuring out the right models and

available compute resources was troublesome. There were many times where the Linux computer used would run out of memory or encounter driver errors that made the system unstable. The only solution was to manually address these issues as they came up, especially for CUDA programs. For sourcing our LLM models, we used Ollama, specifically for models `mistral:7b`, `qwen2.5:0.5b`, and `llama3.2:1b`. Testing of the models sped up drastically with the latter 2 models, as they were both small enough to fit on our GPU's limited VRAM (4GB). `Llama3.2:1b` was the model used in the final code, as `Qwen2.5:0.5b` had the habit of occasionally hallucinating responses in Chinese instead of English, on top of the overall poorer quality of responses. Refer to the Appendix for more specifications of our setup.

For speech-to-text, OpenAI's Whisper package was used, opting for the "tiny" multilingual model. This was to ensure better recognition for non-english words, such as proper nouns like names. The performance difference between the "tiny" and "tiny.en" models was negligible.

For verbalizing the LLM's responses into spoken dialogue, we initially tried using Google's gTTS python package, but this involved manually speeding up playback of the agent's response. This technique produced an unnatural tone of voice when any

speedup/slowdown of speech were applied. We then transitioned to Piper-TTS, which had built-in functionality to change its playback speed, without any degradation in playback quality.

4.2 Deployable Web-App Approach

As the project continued beyond the fall semester, development pivoted to an Azure cloud-based approach. By making use of Azure's Speech Recognizer, integration of ChatGPT models via Azure AI Studio, its own TTS system with playback speed manipulation, this approach was pursued in the hope of being able to deploy the app and conduct user testing remotely. Sadly, this never happened due to difficulties with the Python Web App library Streamlit, which we used for rapid implementation. This was in tandem with Python's poor ability to handle multiple interactions asynchronously. Given these time-consuming challenges, we pivoted back to the local setup, for which we had been steadily back-porting features to in preparation for such a scenario.

4.3 Speech Rate

Common between both approaches is the usage of Myprosody. This library works best with 48kHz .wav files, as this format ensures that the most accurate syllables per second can be extracted from the file. Challenges arose from integrating this package, as its code only outputs values to the terminal. There were also ambivalent error messages on the audio quality that are printed if the program encounters any error while processing the file, which includes file path mismatches, such as trying to use relative paths.

Once setup issues were resolved, we performed preliminary testing with our own speech at varying speeds. These experiments allowed us to come up with the following calculation to make the system vary its speech rate to match the user's speech rate:

$$\alpha = \frac{U_{\text{current}}}{U_{\text{default}}}, \quad S_{\text{new}} = \frac{\alpha + S_{\text{current}}}{2}, \quad \beta = \frac{T}{U_{\text{default}}} * W$$

$$\text{length_scale (needed by PiperTTS)} = \frac{\beta}{S_{\text{new}}}$$

The U variables denote the user speech rates in syllables per second, while S variables denote the speed factor. Based on an average of the current speed factor and a new factor based on how the user's current speech rate U_{current} is in comparison to the default, U_{default} . T is the default speech rate of the PiperTTS. We have to apply the speed factor's inverse to the ratio between the User's and PiperTTS's default speech rate due to how PiperTTS implements speed via a `length_scale` variable, where smaller rationals make the response faster, and larger ones make it slower. The aforementioned T/U_{default} fraction is used to account for the difference between the User's default and the TTS's, and W is a little bit of manual tuning we did to speed up the TTS to match the user's average of 3.0 syllables per second, given a speed factor of 1.0x.

A while-true loop keeps the conversation going, and usage of Langchain packages facilitates continued prompting of the agent, as well as message history. This pipeline does not include online search capabilities.

Both the online model and final local approaches also have built-in logging functionality, and export .csv logs with an ISO format

timestamp, message, and current speed factor as the columns. Log messages record what the user's input speech rate is, what the new speed factors are, and responses for the user and agent.

5 User Testing Setup

Since this work had developed more into a system implementation rather than the proposed user study, the setup and data collected of participants were slightly scaled down to suit the time constraints more appropriately.

In our experiments, we recorded the .csv log files of users' free, unstructured conversations with the agent, each of which lasted for about 3-4 minutes. These log files were used to determine the user and assistant's min, max and average speech rate, as well as percentages on how much faster/slower the adaptive rate response is compared to the agent's default, static rate.

Additionally, qualitative user responses were noted during informal debrief on their experience after they had finished speaking with the agent and had taken a 5-minute break. The interview included space for open-ended feedback, as well as questions on the user's proficiency and history with CAs and voice-assistive technologies (specifically speech-to-text and text-to-speech). Further follow-up questions ensued to further explore the users' experience with and impressions of the agent. Noteworthy user responses and feedback are further discussed in the Results section.

For the experimental setup, all participants were present in a waiting room, and a separate vacant room was set up with a notebook and a laptop for them to use to interact with the agent.

Users could see the transcription of their speech, but could not see the text of the Agent's response. One of us was present to observe in the testing room and direct participants between experiments. Users made use of a headset with a boom microphone to ensure clear audio.

Participants were informed to speak at a natural rate of speech, and that both systems were commercially available in an attempt to avoid biases towards the implemented system, since it was created by one of the observers. Users were not explicitly told that the agent was designed to adapt to their speech rate, and were only told after testing to illicit more informative feedback/responses.

For the informal debrief, all participants were present in a group discussion, allowing each participant to help refine their feedback and comment on each other's experiences.

6 Results

6.1 Caveats & Disclaimers

It bears mentioning that our small participant pool does have some inherent biases, as all of them were UTCS students, biasing towards high technology literacy. Furthermore, it is highly probable, if not certain, that all were uniquely experienced in using AI tools such as LLMs and AI Agents, due to the lasting impacts of COVID-19 online schooling at the start of their studies.

While these results and findings could be applied of a student population, the same cannot be done for a more representative group of the general population of English speakers.

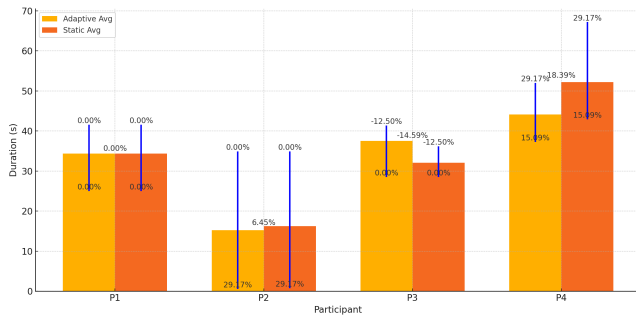


Figure 3: Analysis of Agent Speech Duration. The bars indicate the difference in Average duration for the Adaptive Speech rate (left bar, yellow) and the Static Speech Rate of 3 syllables per second. The blue lines on each bar indicate the min and max percentage difference between the Static and the Adaptive min and max durations, where negative percentages indicate the Adaptive was slower, and positive percentages indicate that it was faster. The percentage in the middle of the two bars is the percentage difference of the average duration, following the same positive/negative scheme as with the min/max values.

6.2 Quantitative Results & Discussion

To determine "quantitatively" whether or not our system met the criteria of improving the efficiency and comprehension of conversation, we will take a look at the metrics that are illustrated in Figure 3, per participant (P1-4). In the case of P2 and P4, the Adaptive SR responses were faster on average, albeit at small margins. This is in line with what would be expected with the calculations the code makes to adapt SR, always erring on the safe side and being slightly slower than the user's speech rate when they start speaking faster. P3 had the exact opposite relationship between the average agent response duration and the percent difference, as the adaptive SR did not slow down as much as the user, still erring on the safe side as well.

At least from a quantitative perspective, there is a small improvement in communication efficiency given an adaptive SR agent. For users like P4 who can speak faster, having a CA that can keep up with you is certainly going to improve the efficiency of the interaction, at least in terms of time. In terms of observations, P4 was generally more energetic than the other participants. That being said, it should be noted that for the average users, like P2 and P1, there would most likely not be any significant improvements in efficiency given an adaptive SR agent.

6.3 Qualitative Feedback & Discussion

The feedback and interview responses from users denote a minimal impact of the adaptive SR agent, with a neutral to slightly negative impression overall. For instance, P4 noted that hearing the adaptive CA speed up, even in its more gradual manner, made the conversation feel more tense, as they felt the need to keep up the pace, and that at a certain point they most likely would have had to slow down further in a "crashing" manner, essentially walking back some of the time savings that they would have gained had the conversation continued. P2 stated they also noticed the speed-up

of the assistance, but didn't state it aloud in their feedback until we revealed that it was intentional. P1, as would be expected for someone who spoke at a consistent rate, didn't have much in terms of input on the adaptability of the CA.

Additionally, users noted that comprehension was generally poorer with the adaptive SR agent than in prior interactions with other conversational agents, as stated by P2 and P4. When we inquired further about these experiences, P4 mentioned how they would often use PDF readers to help comprehend research papers, allowing them to read along with voice-over. P2 mentioned that they often watch YouTube videos at higher speeds using an extension to manipulate the video speed in a granular manner. They also brought up the usage of captions at times as well to aid their comprehension, especially in busier environments. P3 noted that they had noticed the assistant slowing down, but didn't feel that the difference impacted their comprehension in any significant way. This does line up with the results in the work of Choi et al, where slowing down the assistant speaking rate, though it may create a more comfortable experience, didn't significantly improve comprehension and worsened the efficiency of the conversation [2]. Users can typically read faster than they can hear.

The .csv tables used to create Figure 2 are present in the lets-talk-tempo GitHub repository: <https://github.com/AbhinavJayaraman/lets-talk-tempo/tree/final>

7 Future Work & Considerations

There were many aspects of this system that were compromised on for the sake of time, such as participant size, deployment of the web app, and experimentation with other adaptive speech rate equations. Out of these aspects, we would especially like to expand on the scale of our participant experiments, to see what our small sample size captured and missed. Possibly comparing the impact of making the agents responses visible to the user could be fruitful to test to verify the aforementioned impact further.

References

- [1] Danielle Bragg, Cynthia Bennett, Katharina Reinecke, and Richard Ladner. 2018. A Large Inclusive Study of Human Listening Rates. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3173574.3174018>
- [2] Dasom Choi, Daehyun Kwak, Minji Cho, and Sangsu Lee. 2020. "Nobody Speaks that Fast!" An Empirical Study of Speech Rate in Conversational Agents for People with Vision Impairments. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376569>
- [3] Brett Christenson, Christine Ringler, and Nancy J. Sirianni. 2023. Speaking fast and slow: How speech rate of digital assistants affects likelihood to use. *Journal of Business Research* 163 (2023), 113907. <https://doi.org/10.1016/j.jbusres.2023.113907>
- [4] Tomoo Inoue and Wei Liao. 2019. Speech Speed Awareness System Slows Down Native Speaker's Talk. In *Collaboration Technologies and Social Computing: 25th International Conference, CRIWG+CollabTech 2019, Kyoto, Japan, September 4–6, 2019, Proceedings* (Kyoto, Japan). Springer-Verlag, Berlin, Heidelberg, 159–171. https://doi.org/10.1007/978-3-030-28011-6_11



```

homegrown_setup.ipynb > TTS Speech Code, using Piper-TTS. Why? Because Google TTS is not very good at reading numbers, like years.
Generate + Code + Markdown | Interrupt Restart Clear All Outputs Go To Jupyter Variables Outline ...

with console.status("Processing agent audio..", spinner="earth"):
    new_speedup = agent_response_audio(response, user_sr, cur_sr_speedup)
    log_stamp(f"Computed new speedup: {cur_sr_speedup} -> {new_speedup}", cur_sr_speedup)
    cur_sr_speedup = new_speedup
    delete_file("agent.wav")
    # print(f'Agent SR Speed Up: {cur_sr_speedup}')

else:
    console.print(
        "[red]No audio recorded. Please ensure your microphone is working."
    )

except KeyboardInterrupt:
    console.print("\n[red]Exiting...")

console.print("[blue]Session ended.")

[18] 25.7s

... Agent started! Press Ctrl+C to exit.

... Press Enter to start recording, then press Enter again to stop.

...

... You: Who is the rapper or artist known as Hanuman Kind?

...

... Processing agent audio..

```

8 Appendix

8.1 Picture of the Python Notebook with an example user query

8.2 Local Setup Computer Specifications

The system we made use of was an Acer Swift x 14-in Laptop (2021), running Fedora Linux Workstation 41. For improved Wi-Fi stability, the default MT7921 WiFi card was replaced with a better-supported but older Intel 7265 card. Power-saving measures were turned off for the card. The laptop was running off a 256 GB SSD, with 16 GB of RAM. The CPU is the AMD RYZEN 7 5800u with 8 cores and 16 threads, and a max power draw of 28 Watts. The GPU was the Nvidia RTX 3050 TI, with 4 GB of VRAM, with a max power draw of 40 Watts.